

Software Engineering

Declarative prototyping - Exercises -

Exercise 1

- Quicksort with difference lists – declarative prototype:

```
qsd :: [Int] -> [Int] -> [Int]
qsd []      ys = ys
qsd (x : xs) ys =
    let (ls,gs) = partit x xs
    in qsd ls (x : qsd gs ys)
```

```
partit :: Int -> [Int] -> ([Int],[Int])
partit p []      = ([],[ ])
partit p (x:xs) =
    let (ls,gs) = partit p xs
    in  if (x < p) then (x : ls,gs) else (ls,x:gs)
```

```
Main> qsd [1,7,3,12,4,2,5] [0,0,0]
[1,2,3,4,5,7,12,0,0,0]
```

Solution to exercise 1

- Using the Haskell notation, the concept of a difference list can be understood as follows. If 'ys' is a sublist of 'xs'

$$xs = e1:...:en:ys$$

then the **difference list** xs-ys is:

$$xs-ys = [e1,...,en]$$

- **Correctness proof:**

- We prove by induction on the length of 'xs' that
 $(qsd\ xs\ ys) - ys =$ the sorted permutation of xs
- Induction basis: $xs = [] \Rightarrow (qsd\ []\ ys) - ys = ys - ys = []$, which is correct.
- Induction step: follows by noticing that the lists 'ls' and 'gs' computed by the call (partit x xs) are each strictly shorter than (x:xs), therefore the recursive calls of 'qsd' behave correctly.

Solution to exercise 1

• Imperative implementation in C:

```
#include <stdio.h>    /* printf() */
#include <alloc.h>     /* malloc() */
#include <stdlib.h>    /* randomize(), rand() */

typedef struct elem {
    int info;
    elem *next;
} ELEM, *LIST;

void partit(int p, LIST x, LIST *l, LIST *g)
{
    if (x==0) {*l=0; *g=0;}
    else {
        partit(p,x->next,l,g);
        if ((x->info) < p) { x->next = *l;  *l = x;  }
        else { x->next = *g;  *g = x; }
    }
}
```

Solution to exercise 1

- Implementation as C function (an 'in situ' sorting algorithm):

```
LIST qsd(LIST x, LIST y)
{   LIST l,g;
    if (x==0) return y;
    else {
        partit(x->info,x->next,&l,&g);
        x->next = qsd(g,y);
        return (qsd(l,x));
    }
}
```

- Implementation as C procedure (an 'in situ' sorting algorithm):

```
void qsdr(LIST x, LIST *r, LIST y)
{   LIST l,g;
    if (x==0) (*r)=y;
    else {
        partit(x->info,x->next,&l,&g);
        qsdr(g,&(x->next),y);
        qsdr(l,r,x);
    }
}
```

Solution to exercise 1

- Testing the C implementation:

```
void afislist(LIST l)
{
    if (l==NULL) printf("\n");
    else {
        printf("%d ", l->info);
        afislist(l->next);
    }
}

LIST genlist(int lung, int max)
{
    LIST p;
    if (lung==0) return(NULL);
    else {
        p=(LIST)malloc(sizeof(ELEM));
        p->info=rand()%max;
        p->next=genlist(lung-1,max);
        return(p);
    }
}
```

```
void main()
{
    LIST l,r;
    randomize();
    l=genlist(10,10);
    printf("\nl =");afislist(l);
    qsdr(l,&r,0);
    printf("qsdr -> ");
    afislist(r);
    randomize();
    l=genlist(10,10);
    printf("\nl =");afislist(l);
    printf("qsd -> ");
    afislist(qsd(l,0));
}
```

A possible run:

```
l = 8 7 7 1 3 9 6 1 5 0
qsdr -> 0 1 1 3 5 6 7 7 8 9
```

```
l = 8 7 7 1 3 9 6 1 5 0
qsd -> 0 1 1 3 5 6 7 7 8 9
```

Exercise 2

- Lisp-like structure flattening – declarative prototype:

```
data Lisp = Nil | Atom Int | Cons Lisp Lisp
```

```
flat :: Lisp -> [Int] -> [Int]
```

```
flat Nil ys = ys
```

```
flat (Atom n) ys = n:ys
```

```
flat (Cons car cdr) ys = flat car (flat cdr ys)
```

```
lisp :: Lisp
```

```
lisp = Cons (Cons (Cons (Atom 1) Nil)  
               (Cons Nil (Atom 2)))  
          (Cons (Atom 3) (Atom 4))
```

```
Main> flat lisp [0,0,0]
```

```
[1,2,3,4,0,0,0]
```

Solution to exercise 2

- **Correctness proof:**

An easy structural induction

Solution to exercise 2

• Imperative implementation in C:

```
#include <stdio.h>      /* printf() */
#include <alloc.h>      /* malloc() */

typedef struct elem {
    int info;
    elem *next;
} ELEM, *LIST;

typedef enum {atom, cons} SEL;
typedef struct Lisp {
    SEL sel;
    union {
        int atom;
        struct {
            struct Lisp *car;
            struct Lisp *cdr;
        } cons;
    } lisp;
} CEL, *LISP;
```

Solution to exercise 2

```
LIST flat(LISP l,LIST y)
{ LIST t;
    if (l == 0) return y;
    else if ((l->sel) == atom) {
        t = (LIST)malloc(sizeof(CEL));
        t->info = l->lisp.atom;
        t->next = y; return t;
    } else /* (l->sel) == cons */{
        t = flat(l->lisp.cons.cdr,y);
        return flat(l->lisp.cons.car,t);
    }
}

void flatr(LISP l,LIST *r,LIST y)
{ LIST t;
    if (l == 0) *r = y;
    else if ((l->sel) == atom) {
        *r = (LIST)malloc(sizeof(CEL));
        (*r)->info = l->lisp.atom; (*r)->next = y;
    } else /* (l->sel) == cons */ {
        flatr(l->lisp.cons.cdr,&t,y);
        flatr(l->lisp.cons.car,r,t);
    }
}
```

Solution to exercise 2

- Testing the C implementation:

```
LISP Nil()
{
    return 0;
}

LISP Atom(int a)
{ LISP l;
  l = (LISP) malloc(sizeof(CEL));
  l->sel = atom;
  l->lisp.atom=a;
  return l;
}

LISP Cons(LISP car,LISP cdr)
{ LISP l;
  l = (LISP)malloc(sizeof(CEL));
  l->sel = cons;
  l->lisp.cons.car = car;
  l->lisp.cons.cdr = cdr;
  return l;
}
```

```
void show(LISP l)
{
    if (l==0) printf("Nil");
    else if ((l->sel) == atom)
        printf("%d",l->lisp.atom);
    else if ((l->sel) == cons) {
        printf("(");
        show(l->lisp.cons.car);
        printf(" ");
        show(l->lisp.cons.cdr);
        printf(")");
    }
}

LIST lcons(int a,LIST l)
{ LIST c;
  c = (LIST) malloc(sizeof(ELEM));
  c -> info = a;
  c -> next = l;
  return c;
}
```

Solution to exercise 2

- Testing the C implementation:

```
void main()
{  LISP l; LIST r,t;
    t = lcons(0,lcons(0,lcons(0,0)));
    l = Cons(Cons(Cons(Atom (1),Nil()),Cons(Nil(),Atom(2))),
              Cons(Atom(3),Atom(4)));

    printf("\n"); show(l);
    printf("\nflat ->"); afislist(flat(l,0));
    l = Cons(Cons(Cons(Atom (1),Nil()),Cons(Nil(),Atom(2))),
              Cons(Atom(3),Atom(4)));

    printf("\n"); show(l);
    printf("\nflatr ->");
    flatr(l,&r,t); afislist(r);
}
```

- At execution you get:

```
((1 Nil) (Nil 2)) (3 4)
flat -> 1 2 3 4
((1 Nil) (Nil 2)) (3 4)
flatr -> 1 2 3 4 0 0 0
```

Exercise 3

- Binary search tree merging using difference lists:

```
data Tree = NIL | T Tree Int Tree
flatt :: Tree -> [Int] -> [Int]
flatt NIL      ys = ys
flatt (T l n r) ys = flatt l (n : flatt r ys)

merge :: Tree -> Tree -> [Int] -> [Int]
merge NIL t ys = flatt t ys
merge t NIL ys = flatt t ys
merge (T NIL n1 r1) (T NIL n2 r2) ys =
    if (n1 < n2) then n1:merge r1 (T NIL n2 r2) ys
                  else n2:merge (T NIL n1 r1) r2 ys
merge (T (T l11 ln1 lr1) n1 r1) t2 ys =
    merge (T l11 ln1 (T lr1 n1 r1)) t2 ys
merge t1 (T (T l12 ln2 lr2) n2 r2) ys =
    merge t1 (T l12 ln2 (T lr2 n2 r2)) ys

t1 = T (T NIL 1 (T NIL 3 NIL)) 5 (T NIL 7 NIL)
t2 = T (T NIL 2 NIL) 4 (T (T NIL 6 NIL) 8 NIL)
Main> merge t1 t2 [0,0]
[1,2,3,4,5,6,7,8,0,0,0]
```

Solution to exercise 3

- Correctness proof (for 'merge', the proof for 'flatt' was given in the course notes):
 - We proceed by induction on $c : \text{Tree} \times \text{Tree} \rightarrow \mathbb{N} \times \mathbb{N}$
 - $c(t_1, t_2) = (u(t_1) + u(t_2), v(t_1) + v(t_2))$ (the elements of $\mathbb{N} \times \mathbb{N}$ are **ordered lexicographically**)
 - $u(\text{NIL}) = 0, \quad u(T \mid n \mid r) = 1 + u(l) + u(r)$
 - $v(\text{NIL}) = 0, \quad v(T \mid n \mid r) = 1 + v(l)$
 - We prove the following claim (under the assumption that t_1, t_2 are binary search trees):
 - $(\text{merge } t_1 \ t_2 \text{ } ys) - ys =$ the sorted permutation of the list of nodes in t_1 and t_2
 - **Ind. basis:** $c(t_1, t_2) = (0, 0) \Leftrightarrow t_1 = \text{NIL}, t_2 = \text{NIL}$; in this case we have:
 - $(\text{merge } \text{NIL } \text{NIL } ys) - ys = ys - ys = []$ (correct)
 - **Ind. step:** – there are several subcases
 - $(\text{merge } \text{NIL } t_2 \text{ } ys) - ys = (\text{flatt } t_2 \text{ } ys) - ys =$ the sorted permutation of the list of nodes in t_2 ($\Leftarrow$ the correctness of 'flatt', t_2 is a binary search tree)
 - $(\text{merge } \text{NIL } t_2 \text{ } ys) - ys = (\text{flatt } t_2 \text{ } ys) - ys$ (idem)
 - $(\text{merge } (T \mid \text{NIL } n_1 \mid r_1) \ (T \mid \text{NIL } n_2 \mid r_2) \text{ } ys) - ys =$ the sorted permutation of the list of nodes in t_1 and t_2 , because (let $t_1 = (T \mid \text{NIL } n_1 \mid r_1), t_2 = (T \mid \text{NIL } n_2 \mid r_2)$)
 - If $(n_1 < n_2)$ then n_1 is the smallest node of t_1 and t_2 ;
 - in addition $(\text{merge } r_1 \ (T \mid \text{NIL } n_2 \mid r_2) \text{ } ys) - ys =$ the sorted permutation of the nodes in t_1 and t_2 because $(u(t_1) = 1 + u(r_1) > u(r_1))$
 - $c(t_1, t_2) = (u(t_1) + u(t_2), v(t_1) + v(t_2)) > (u(r_1) + u(t_2), v(r_1) + v(t_2))$
 - The case $(n_1 \geq n_2)$ is symmetric
 - Let $t_1 = (T \mid (T \mid l_1 \mid l_1) \mid n_1 \mid r_1), t_1' = (T \mid l_1 \mid l_1 \mid (T \mid l_1 \mid n_1 \mid r_1))$. Then
 - $(\text{merge } t_1 \ t_2 \text{ } ys) - ys = (\text{merge } t_1' \ t_2 \text{ } ys) - ys =$ the sorted permutation of the list of nodes in t_1 and t_2 , because the list of nodes in t_1 and $t_2 =$ the list of nodes in t_1' and t_2 , and
 - By the ind.hyp. $(\text{merge } t_1' \ t_2 \text{ } ys) - ys =$ the sorted permutation of the list of nodes in t_1' and t_2 ; indeed $c(t_1, t_2) = (u(t_1) + u(t_2), v(t_1) + v(t_2)) > (u(t_1') + u(t_2), v(t_1') + v(t_2)) = c(t_1', t_2)$, because $u(t_1) = u(t_1')$, but $v(t_1) > v(t_1')$.
 - The case $(\text{merge } t_1 \ (T \mid (T \mid l_2 \mid l_2) \mid n_2 \mid r_2) \text{ } ys)$ is symmetric.

Solution to exercise 3

- **Imperative implementation in C:**

```
#include <stdio.h>    /* printf() */
#include <alloc.h>     /* malloc() */
```

```
typedef struct elem {
    int info;
    struct elem *next;
} ELEM, *LIST;
```

```
typedef struct nod {
    int info;
    struct nod *left, *right;
} NODE, *TREE;
```

Solution to exercise 3

```
LIST flatt (TREE t, LIST y)
{ LIST x;
  if (t == 0) return y;
  else {
    x = (LIST)malloc(sizeof(ELEM));
    x->info = t->info;
    x->next = flatt(t->right, y);
    return(flatt(t->left, x));
  }
}
```

```
void flattr(TREE t, LIST *x, LIST y)
{ LIST z;
  if (t == 0) (*x)=y;
  else {
    z = (LIST)malloc(sizeof(ELEM));
    z->info = t->info;
    flattr(t->right, &(z->next), y);
    flattr(t->left, x, z);
  }
}
```

```
LIST cons(int i, LIST n)
{ LIST l;
  l = (LIST)malloc(sizeof(ELEM));
  l->info= i;
  l->next = n;
  return l;
}
```


Solution to exercise 3

```
LIST merge(TREE t1, TREE t2, LIST y)
{ LIST x; TREE p;
  if (t1 == 0) return (flatt(t2,y));
  else if (t2 == 0) return (flatt(t1,y));
  else if ((t1->left == 0) && (t2->left) == 0) {
    if (t1->info < t2->info)
      cons(t1->info, merge(t1->right, t2, y));
    else
      cons(t2->info, merge(t1, t2->right, y));
  } else if (t1->left != 0) {
    p = t1;
    t1 = p->left; p->left = t1->right;
    t1->right = p;
    return (merge(t1, t2, y));
  } else {
    p = t2;
    t2 = p->left; p->left = t2->right;
    t2->right = p;
    return (merge(t1, t2, y));
  }
}
```

Solution to exercise 3

```
void merger(TREE t1, TREE t2, LIST *x, LIST y)
{ TREE p;
  if (t1 == 0) flattr(t2, x, y);
  else if (t2 == 0) flattr(t1, x, y);
  else if ((t1->left == 0) && (t2->left) == 0) {
    (*x) = (LIST) malloc(sizeof(ELEM));
    if (t1->info < t2->info) {
      (*x)->info = t1->info;
      merger(t1->right, t2, &((*x)->next), y);
    } else {
      (*x)->info = t2->info;
      merger(t1, t2->right, &((*x)->next), y);
    }
  } else if (t1->left != 0) {
    p = t1; t1 = p->left; p->left = t1->right;
    t1->right = p;
    merger(t1, t2, x, y);
  } else {
    p = t2; t2 = p->left; p->left = t2->right;
    t2->right = p;
    merger(t1, t2, x, y);
  }
}
```

Solution to exercise 3

```
TREE NIL()
{
    return 0;
}

TREE T(TREE l,int node,TREE r)
{
    TREE t;
    t = (TREE)malloc(sizeof(NODE));
    t->info = node;
    t->left = l;
    t->right = r;
    return t;
}

void afislist(LIST l)
{
    if (l==0) printf("\n");
    else {
        printf("%d ",l->info);
        afislist(l->next);
    }
}
```

Solution to exercise 3

```
void main()
{ TREE t1,t2; LIST y,x;
  t1 = T (T(NIL(),1,T(NIL(),3,NIL())) ,5,T(NIL(),7,NIL()));
  t2 = T (T(NIL(),2,NIL()) ,4,T(T(NIL(),6,NIL()),8,NIL()));
  y = cons(100,cons(100,cons(100,0)));
  printf("\nStart\n"); afislist(y); printf("\nmerge=>");
  afislist(merge(t1,t2,y));
  t1 = T (T(NIL(),1,T(NIL(),3,NIL())) ,5,T(NIL(),7,NIL()));
  t2 = T (T(NIL(),2,NIL()) ,4,T(T(NIL(),6,NIL()),8,NIL()));
  y = cons(100,cons(100,cons(100,0)));
  afislist(y); printf("\nmerger=>");
  merger(t1,t2,&x,y);
  afislist(x); printf("\nStop");
}
/* Result obtained at run time:
Start
100 100 100
merge=> 1 2 3 4 5 6 7 8 100 100 100
100 100 100
merger=> 1 2 3 4 5 6 7 8 100 100 100
Stop
*/
```